
Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 6
End-to-End Study of RNN, LSTM and GRU for
Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Report basis. This report is prepared from the supplied Experiment 6 specification and the supplied executed Jupyter notebook. Numerical values, plots and sample predictions below are the observed outputs of that notebook execution; they are not assumed values.

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The primary sequence task uses the UCI Human Activity Recognition Using Smartphones dataset. Temporal input sequences are prepared in (N, T, F) form, visualized, normalized and classified using SimpleRNN, LSTM and GRU models. Their predictive performance, convergence behaviour, confusion matrices, parameter counts and training times are compared. A second pipeline uses MobileNetV2 features from video frames followed by LSTM or GRU. Finally, a synthetic reversal task demonstrates encoder–decoder sequence-to-sequence learning.

2. Learning Outcomes

The experiment demonstrates the following outcomes:

- sequential data representation using (N, T, F) ;
- operation of a Vanilla RNN and recurrent hidden states;
- BPTT and the vanishing/exploding-gradient problem;
- sequence classification using SimpleRNN, LSTM and GRU;
- quantitative comparison using accuracy, precision, recall, F1-score, parameters and training time;
- interpretation of learning curves and confusion matrices;
- CNN-based spatial feature extraction from video frames;
- temporal modelling of CNN features with LSTM/GRU;
- encoder–decoder sequence-to-sequence learning; and
- the relationship among predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary dataset: UCI Human Activity Recognition Using Smartphones (HAR). The dataset contains six activities: WALKING, WALKING UPSTAIRS, WALKING DOWNSTAIRS, SITTING, STANDING and LAYING. The experiment uses the raw inertial signal files rather than only the pre-computed 561-feature vectors. Each window has 128 temporal measurements and nine channels: three body-acceleration axes, three body-gyroscope axes and three total-acceleration axes.

The desired representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset reference specified in the laboratory material is <https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>.

In the executed notebook, the original training and testing partitions were loaded and contained 7352 and 2947 windows respectively, giving 10299 windows after concatenation. The combined data were then split stratified into 70% training, 15% validation and 15% testing using random state 42. The resulting tensors were:

$$X_{train} = (7209, 128, 9), \quad X_{val} = (1545, 128, 9), \quad X_{test} = (1545, 128, 9).$$

Label	Activity	Train	Validation	Test
0	WALKING	1205	258	259
1	WALKING UPSTAIRS	1081	232	231
2	WALKING DOWNSTAIRS	984	211	211
3	SITTING	1244	266	267
4	STANDING	1334	286	286
5	LAYING	1361	292	291

Input representation

For one sample,

$$X_i \in \mathbb{R}^{128 \times 9},$$

and for a batch of 32,

$$X_{batch} \in \mathbb{R}^{32 \times 128 \times 9}.$$

Here N denotes the number of sequences, $T = 128$ the number of time steps and $F = 9$ the number of sensor channels.

4. Overall Experimental Pipeline

The sequence-classification pipeline implemented in the notebook is:



All recurrent models used the same general classifier structure: 32 recurrent units, dropout 0.2, Dense(16, ReLU), Dense(6, softmax), Adam with learning rate 10^{-3} , batch size 32 and 30 epochs.

5. Preprocessing

The raw inertial signals were loaded from the nine signal files and stacked into $(N, 128, 9)$. Labels were converted from the original 1–6 representation to 0–5. Standardization was applied by fitting a `StandardScaler` only on the training data after reshaping the training tensor to $(N \cdot 128, 9)$; the fitted transformation was then applied to validation and test data and reshaped back.

The resulting representation was:

```

Training   : (7209, 128, 9)
Validation : (1545, 128, 9)
Testing    : (1545, 128, 9)
Classes    : 6
Time steps : 128
Features   : 9
  
```

6. Temporal Data Visualization

Representative sequences from WALKING, WALKING UPSTAIRS and SITTING were plotted using Body Acc X, Body Gyro X and Total Acc X.

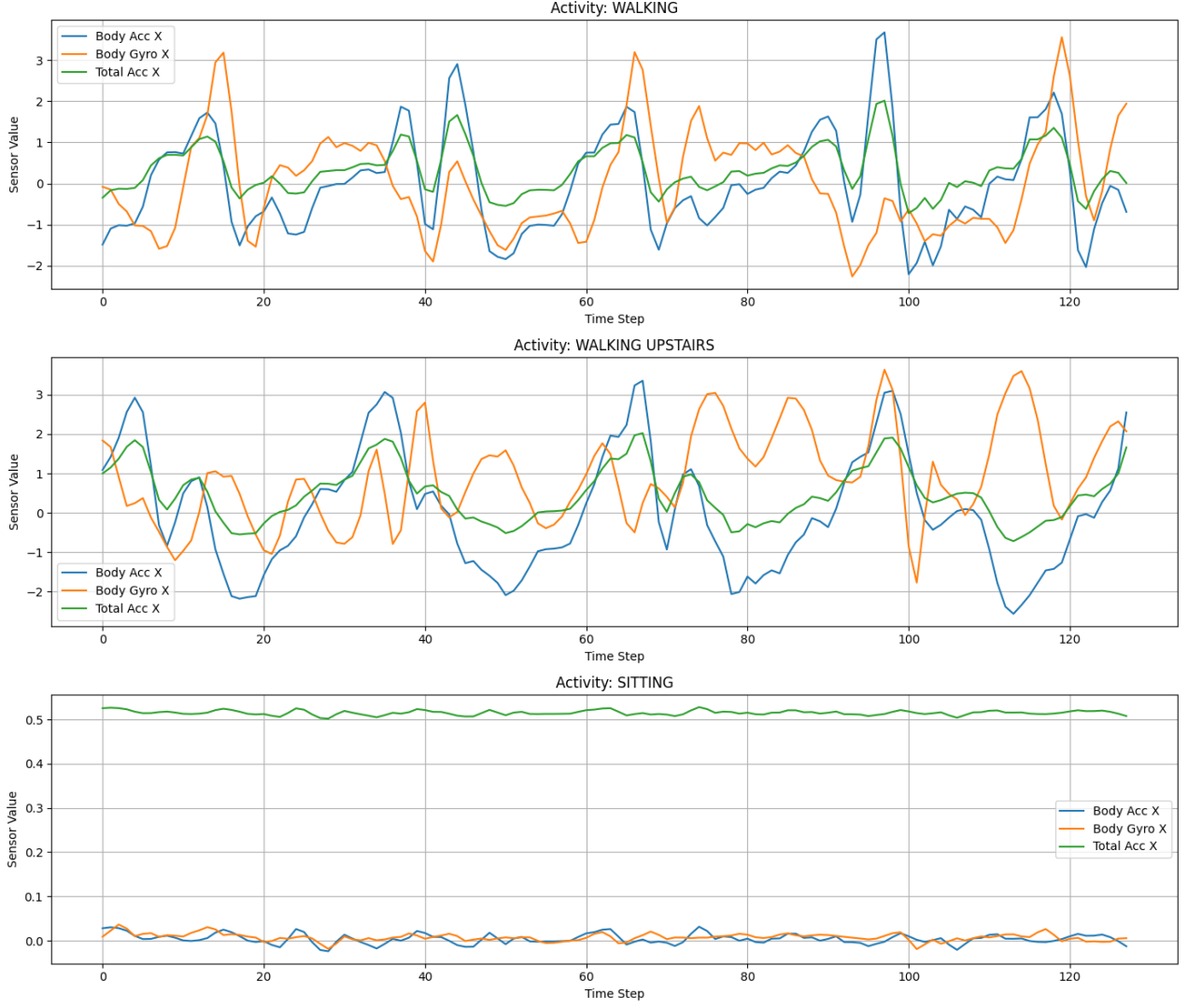


Figure 1: Temporal sensor signals for three representative activity classes.

Inference. The walking activities show visibly varying and periodic sensor patterns, whereas SITTING is much more stable, especially in total acceleration. The walking and walking-upstairs signals share dynamic changes but differ in their detailed temporal patterns. The ordering of samples is important because the recurrent models learn how sensor values evolve over time rather than treating each measurement as an independent tabular feature.

7. Vanilla RNN Architecture

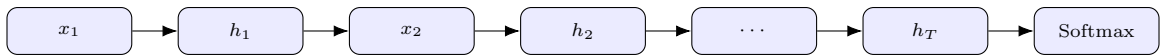
A Vanilla RNN updates its hidden state at every time step according to

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h),$$

and its output may be written as

$$y_t = g(W_y h_t + b_y).$$

For sequence classification, the final hidden representation is passed to the classifier.



The recurrent weights are shared across time steps. This parameter sharing allows the same transition rule to process the complete sequence.

8. Backpropagation Through Time

When the RNN is unrolled, the loss depends on hidden states across the sequence. BPTT propagates the error backward through those recurrent steps. Repeated multiplication of derivatives can make gradient magnitudes extremely

small or extremely large.

Vanishing and exploding gradients

Conceptually, the vanishing-gradient problem can be expressed as

$$\left\| \frac{\partial h_t}{\partial h_{t-k}} \right\| \rightarrow 0,$$

while exploding gradients correspond to very large gradient magnitudes. These effects make long-term dependencies difficult for a conventional RNN and also reduce parallelism because recurrent computation proceeds sequentially.

Numerical exercise

For $x_1 = 0.5$, $x_2 = 0.7$, $x_3 = 0.2$, $h_0 = 0$, $W_x = 0.5$, $W_h = 0.8$ and $b = 0.1$,

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

Manual evaluation gives approximately

$$h_1 = \tanh(0.35) \approx 0.3364,$$

$$h_2 = \tanh(0.45 + 0.8(0.3364)) \approx 0.5238,$$

$$h_3 = \tanh(0.10 + 0.8(0.5238)) \approx 0.4860.$$

These values provide a simple numerical illustration of recurrent state propagation.

9. RNN Implementation

The implemented Vanilla RNN used SimpleRNN(32), Dropout(0.2), Dense(16, ReLU) and Dense(6, softmax). Adam was used with learning rate 10^{-3} , sparse categorical cross-entropy, batch size 32 and 30 epochs.

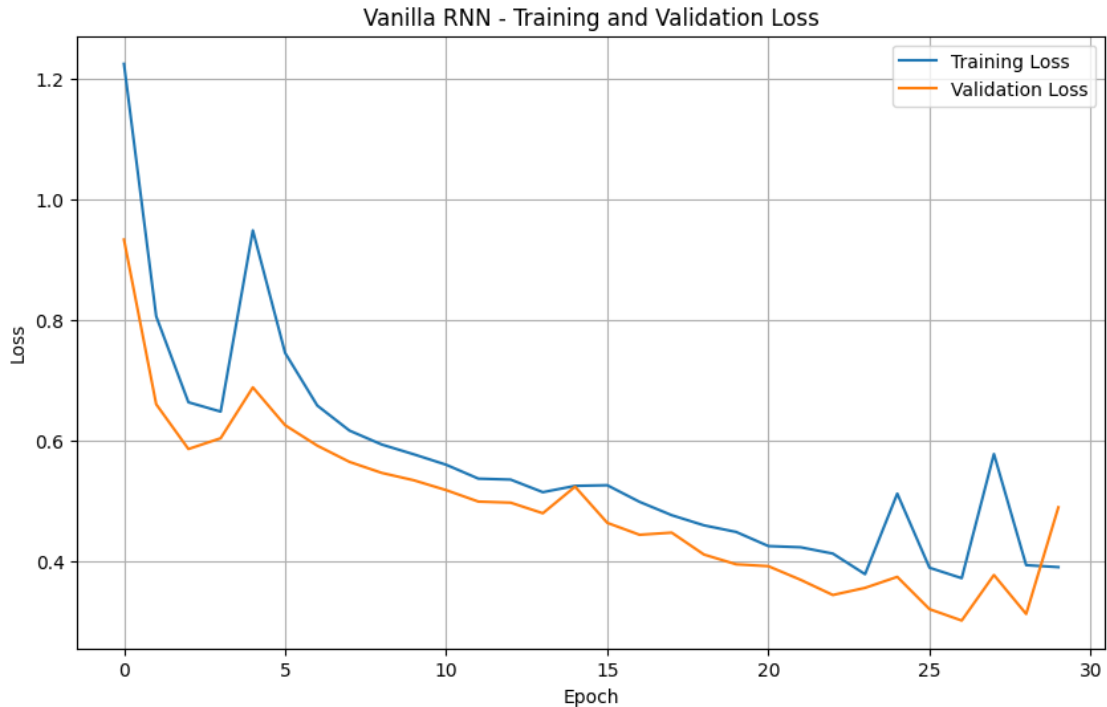


Figure 2: Vanilla RNN training and validation loss.

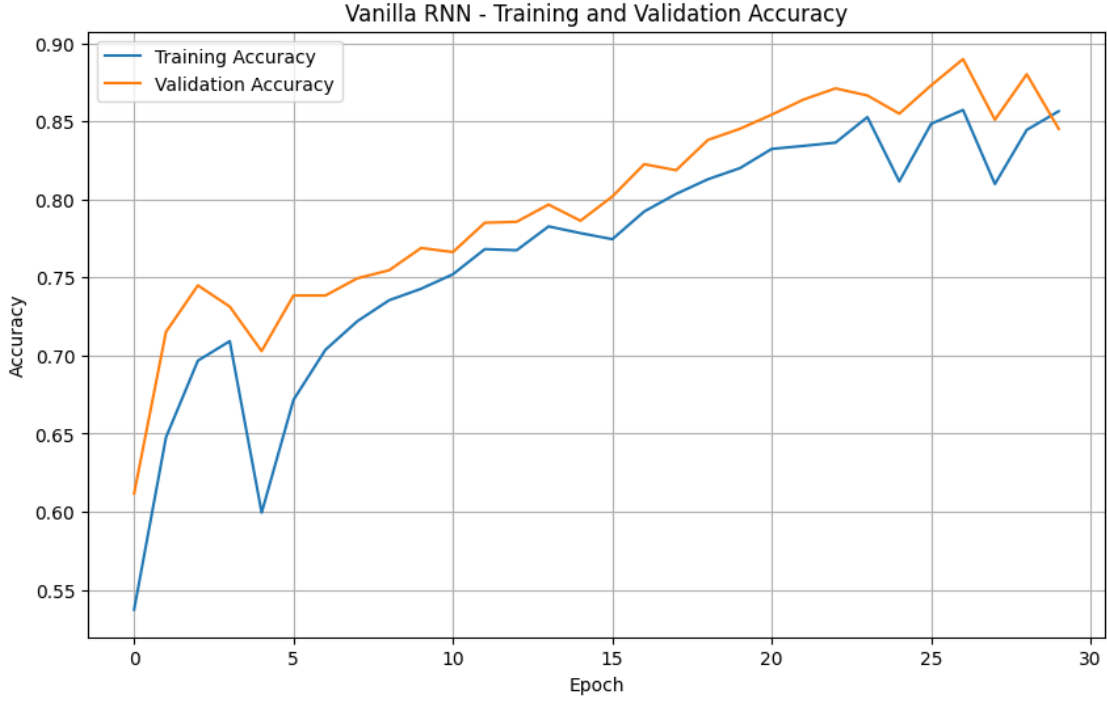


Figure 3: Vanilla RNN training and validation accuracy.

Observed result. Test loss was 0.4768 and test accuracy was 84.08%. Macro precision, recall and F1-score were 83.65%, 83.50% and 83.49%, respectively. The model contained 1974 trainable parameters and required 80.37 seconds for training.

Convergence inference. The RNN generally improved over the 30 epochs, but both loss and accuracy curves show fluctuations. The validation accuracy reached high values during training but ended below some earlier peaks, indicating that optimization was less stable than for the gated recurrent models.

10. LSTM Architecture

LSTM introduces a memory cell and gates that control information flow. The gates are

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

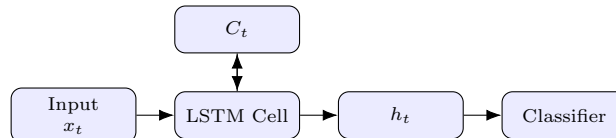
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o),$$

and the candidate cell state is

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C).$$

The cell state and hidden state are

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t, \quad h_t = o_t \tanh(C_t).$$



The cell state provides a pathway for information to persist across many time steps, while the gates learn when information should be forgotten, written or exposed.

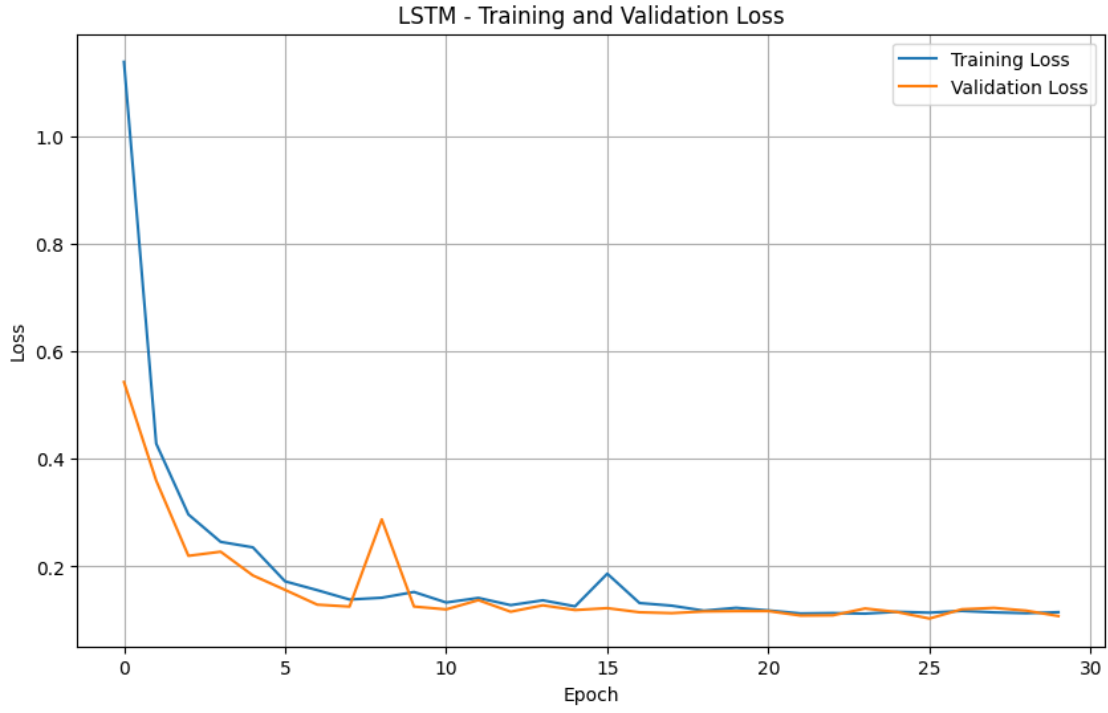


Figure 4: LSTM training and validation loss.

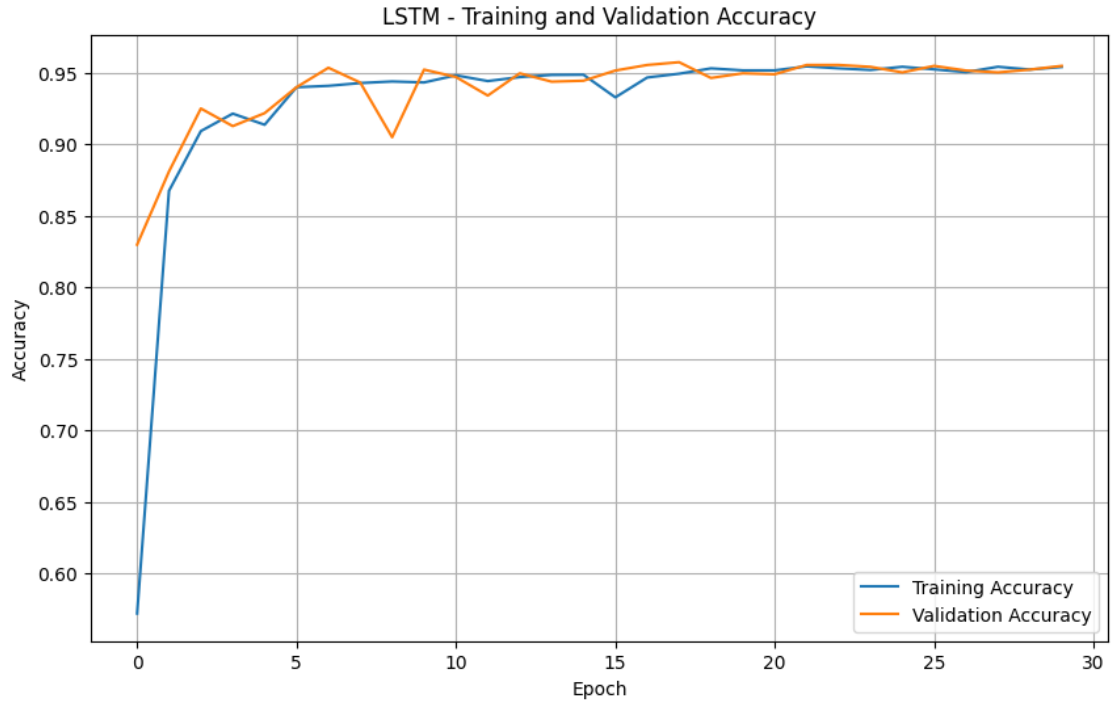


Figure 5: LSTM training and validation accuracy.

Observed result. The LSTM achieved 95.02% test accuracy, 95.37% macro precision, 95.38% macro recall and 95.34% macro F1-score. It used 6006 trainable parameters and required 79.51 seconds for training.

Convergence inference. The LSTM reached high validation accuracy quickly and its loss reduced sharply in the early epochs. The curves subsequently stabilized around the high-accuracy region, showing the benefit of gated memory for this temporal classification problem.

11. GRU Architecture

GRU uses two principal gates:

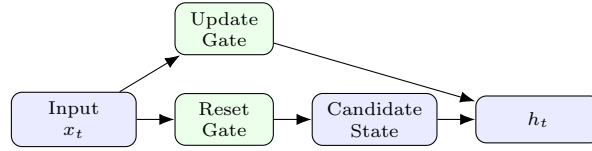
$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z), \quad r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h),$$

and the updated hidden state is

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



GRU has an update gate and reset gate but does not maintain a separate cell state. This gives it a simpler gating structure than LSTM.

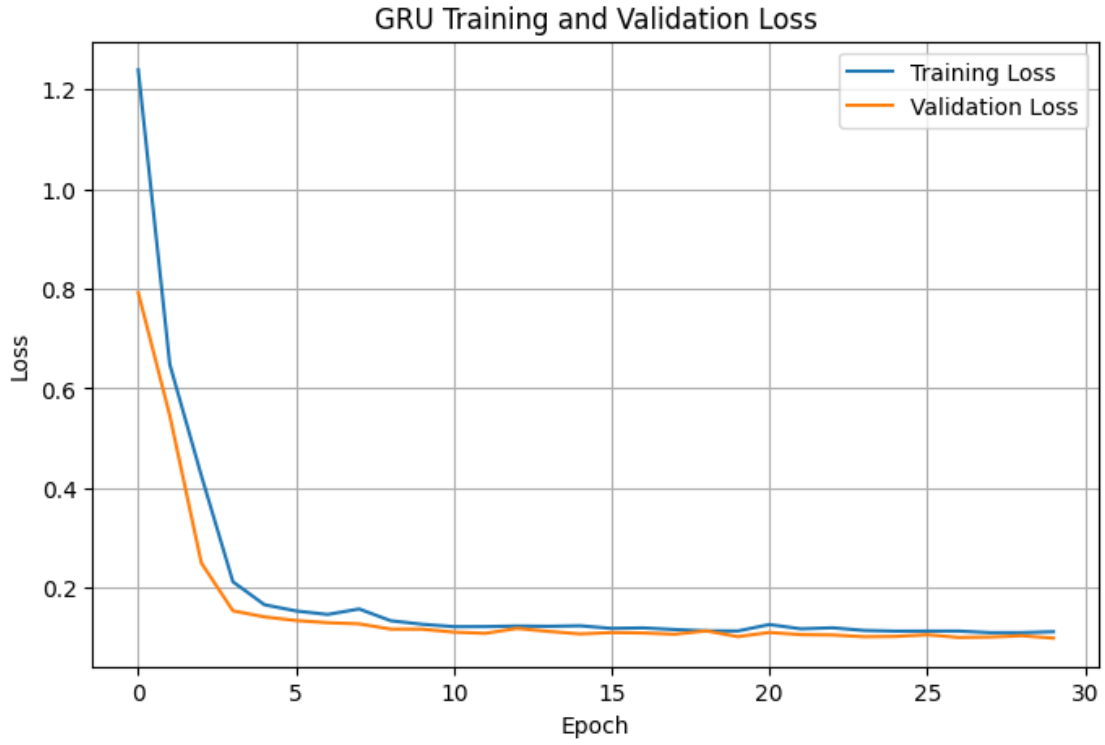


Figure 6: GRU training and validation loss.

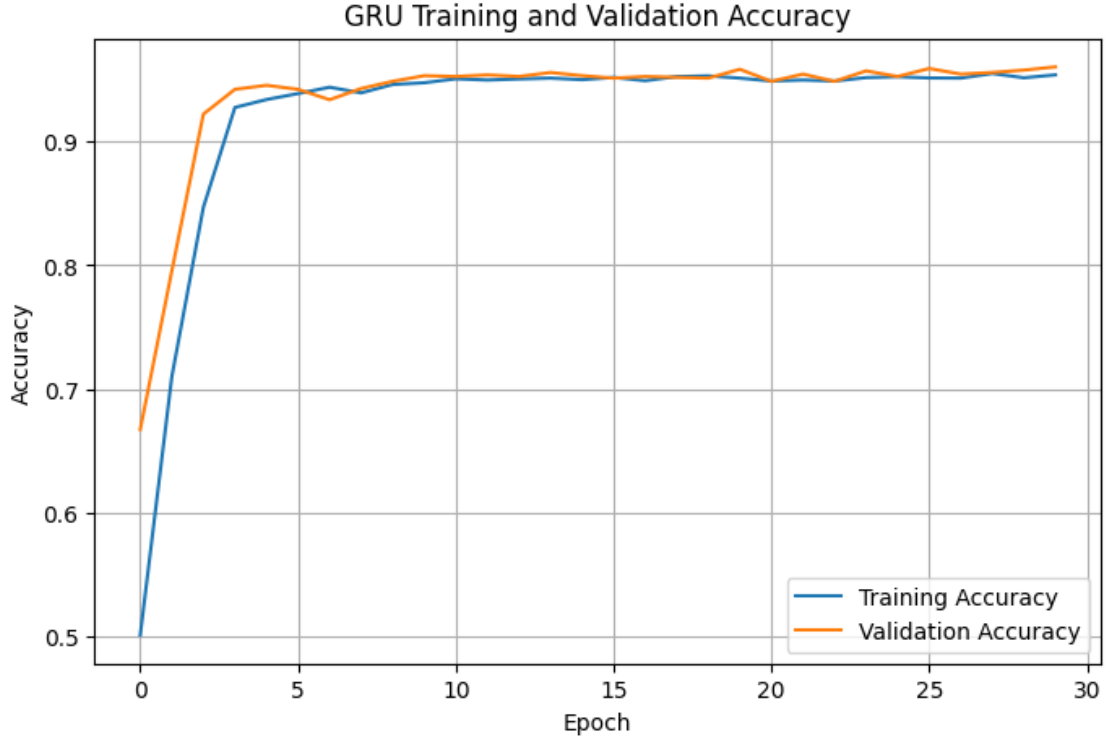


Figure 7: GRU training and validation accuracy.

Observed result. The GRU achieved 95.08% test accuracy, 95.46% macro precision, 95.41% macro recall and 95.38% macro F1-score. It used 4758 trainable parameters and required 73.99 seconds.

Convergence inference. The GRU loss falls rapidly during the first few epochs and then remains close to a low, stable value. Accuracy similarly reaches a high level early and remains stable, indicating effective temporal modelling with a smaller recurrent state structure than LSTM.

12. LSTM and GRU Implementation

Only the recurrent layer was changed while keeping the surrounding classifier and training conditions consistent.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

13. Required Training Plots

The training and validation curves for all three recurrent models were generated and are included in Sections 9–11. The curves provide direct evidence of convergence and generalization behaviour. The RNN curves show larger fluctuations, while the gated models converge more rapidly and remain at substantially higher validation accuracy.

14. Performance Evaluation

The independent test split was evaluated using accuracy, macro precision, macro recall, macro F1-score, parameter count and training time.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Parameters	Time (s)
RNN	84.08%	83.65%	83.50%	83.49%	1,974	80.37
LSTM	95.02%	95.37%	95.38%	95.34%	6,006	79.51
GRU	95.08%	95.46%	95.41%	95.38%	4,758	73.99

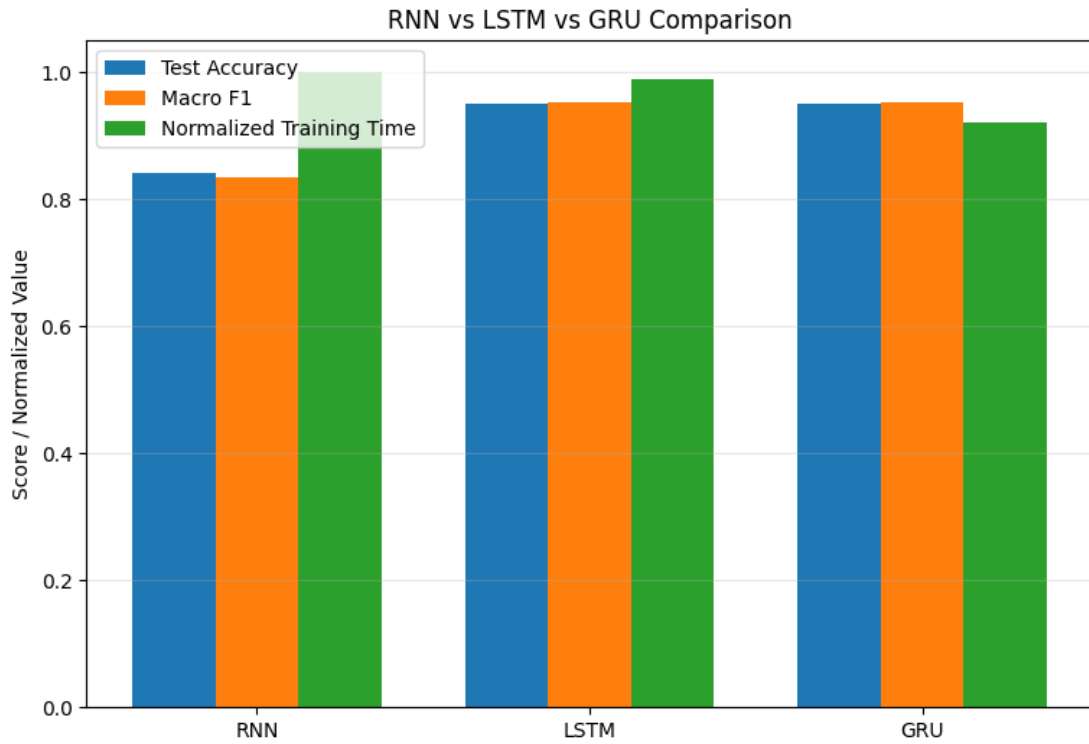


Figure 8: RNN, LSTM and GRU comparison using accuracy, macro F1 and normalized training time.

15. Confusion Matrix Analysis

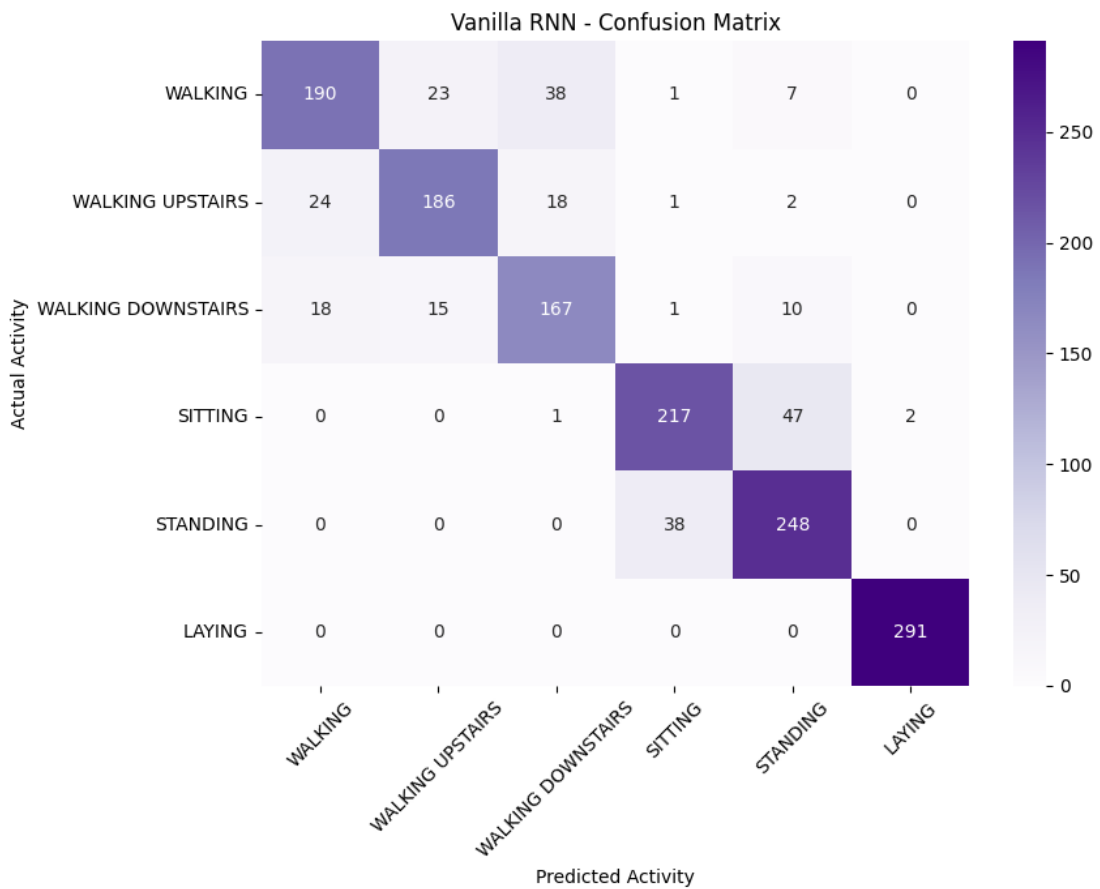


Figure 9: Vanilla RNN confusion matrix.

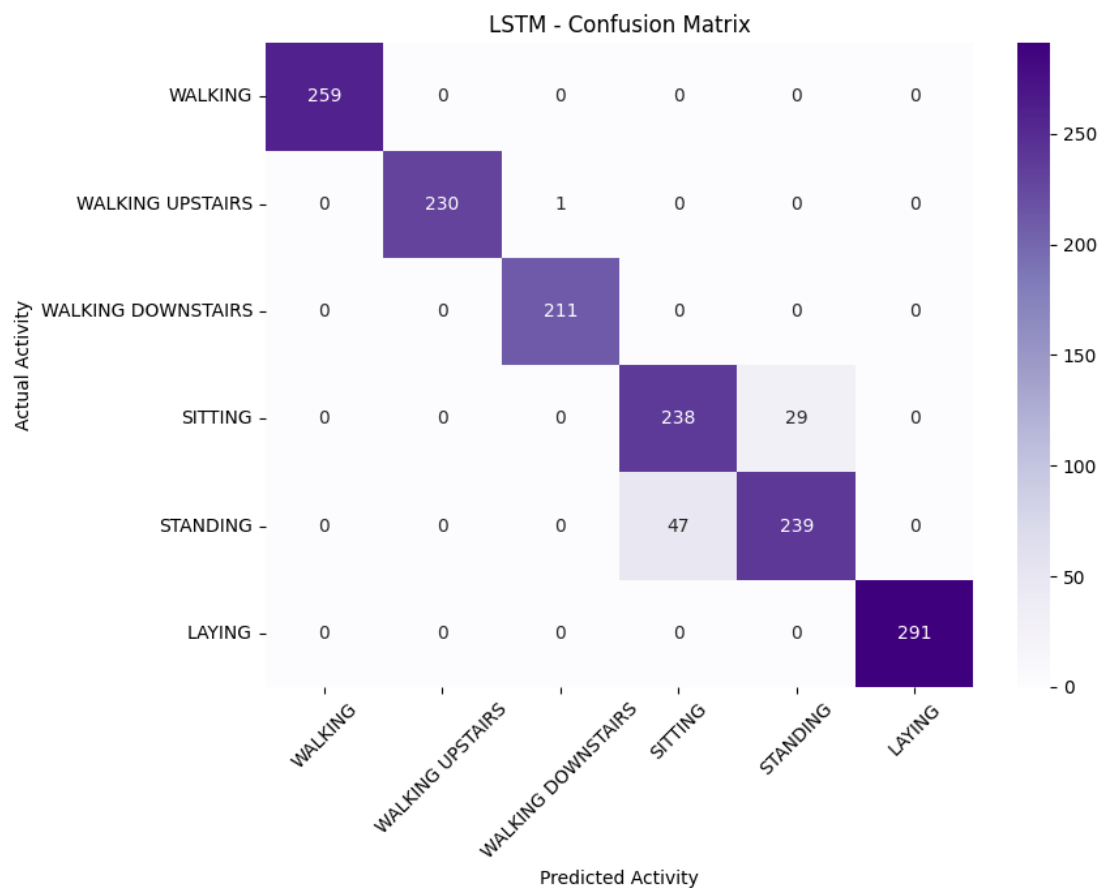


Figure 10: LSTM confusion matrix.



Figure 11: GRU confusion matrix.

RNN inference. LAYING is recognized perfectly in the observed test matrix. The main errors are among dynamic walking activities and between SITTING and STANDING. In particular, 47 SITTING samples are predicted as STANDING and 38 STANDING samples are predicted as SITTING. WALKING also shows confusion with WALKING DOWNSTAIRS and WALKING UPSTAIRS.

LSTM inference. LAYING and WALKING are recognized without observed off-diagonal errors. The dominant remaining confusion is SITTING versus STANDING: 29 SITTING samples are predicted as STANDING and 47 STANDING samples are predicted as SITTING. Only one WALKING UPSTAIRS sample is confused with WALKING DOWNSTAIRS.

GRU inference. The GRU matrix is similarly strong. The largest residual confusion is again between SITTING and STANDING, with 24 SITTING samples predicted as STANDING and 47 STANDING samples predicted as SITTING. LAYING has 291 correct predictions. Walking-related classes have only a small number of errors.

16. RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time	80.37 s	79.51 s	73.99 s
Test macro F1	83.49%	95.34%	95.38%

The observed results show a substantial performance gap between the Vanilla RNN and the two gated architectures. LSTM uses the largest number of parameters in this experiment, while GRU uses fewer parameters and also had the lowest training time among the three recurrent models. The comparison illustrates the trade-off between predictive performance, model complexity and training cost without changing the main classifier settings.

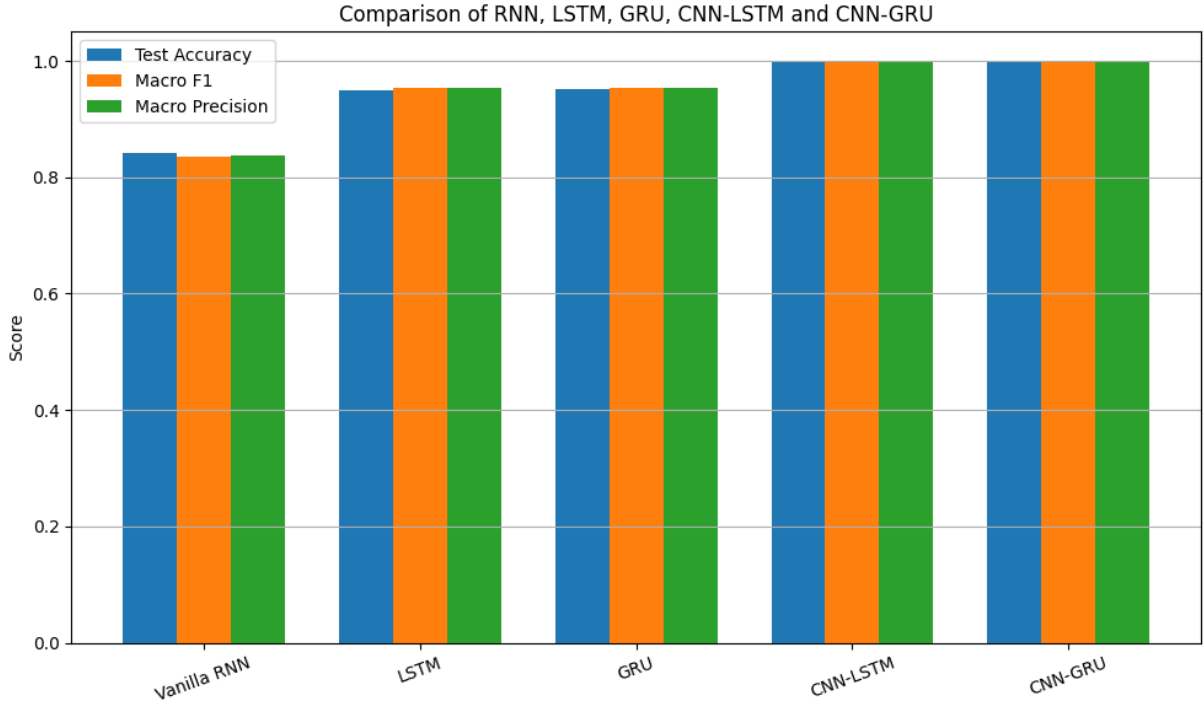


Figure 12: Consolidated comparison of recurrent and CNN-recurrent models.

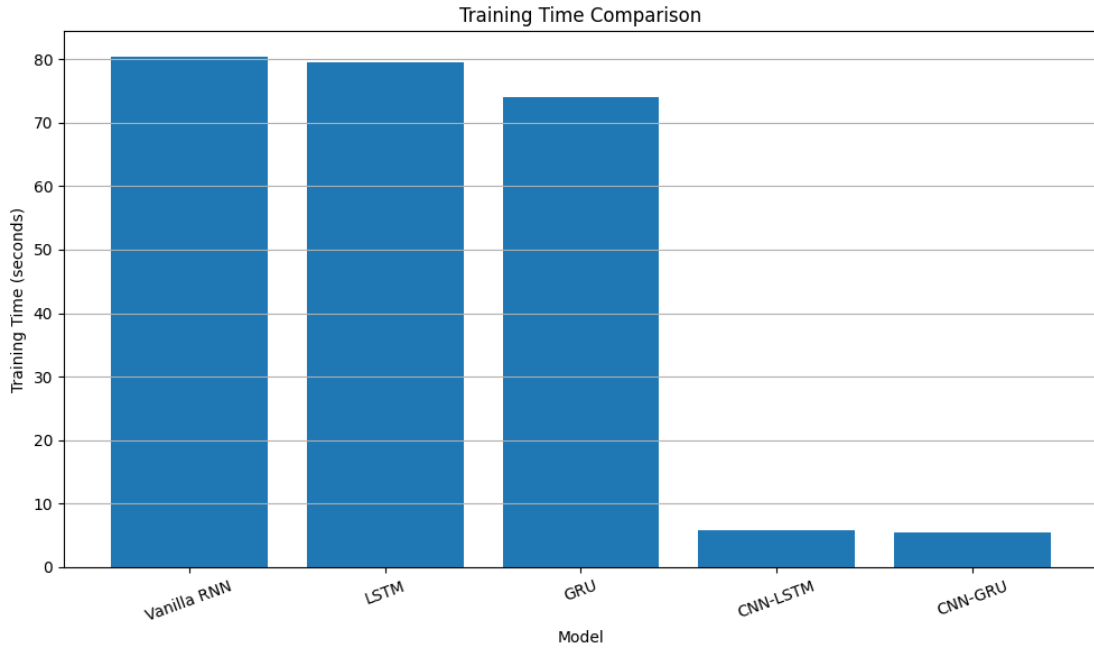


Figure 13: Observed training-time comparison for the consolidated models.

17. Effect of Sequence Length

The recurrent experiment was repeated using $T \in \{32, 64, 128\}$ by truncating the normalized input sequences consistently.

T	RNN Acc.	RNN F1	LSTM Acc.	LSTM F1	GRU Acc.	GRU F1
32	86.73%	86.48%	93.33%	93.63%	93.33%	93.52%
64	87.64%	87.34%	94.37%	94.65%	94.50%	94.83%
128	89.39%	89.45%	86.34%	85.46%	94.56%	94.93%

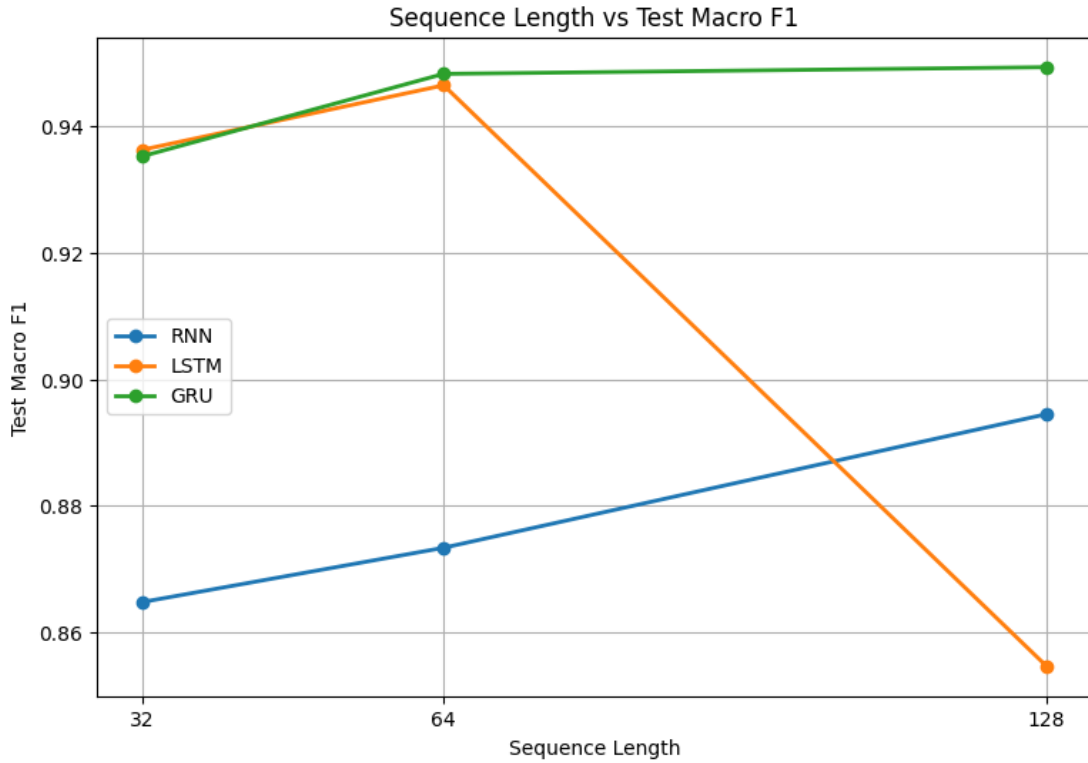
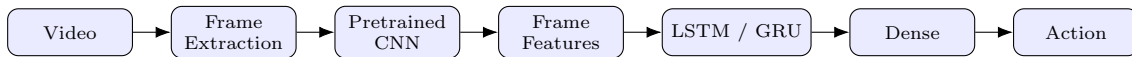


Figure 14: Sequence length versus test macro F1-score.

Inference. RNN performance increases consistently as sequence length increases, with its highest observed F1 at $T = 128$. GRU also improves from 32 to 128 and reaches its highest observed F1 at 128. LSTM improves from 32 to 64 but drops at 128 in this particular execution. Therefore, the amount of temporal context does not have a universally monotonic effect; the observed response depends on the recurrent architecture and training run.

18. Video Understanding Using CNN and RNN

A video is treated as a sequence of frames. A CNN extracts spatial information from each frame, while a recurrent network models temporal relationships between the extracted frame features.



For the executed notebook, a small UCF101 subset was used with three classes: ApplyEyeMakeup, Archery and BabyCrawling. Ten frames were uniformly sampled from each video and resized to $224 \times 224 \times 3$. A total of 121 videos were processed, producing 84 training, 18 validation and 19 testing videos.

19. Expected Video Input and Output

The frame tensor for one video is $(10, 224, 224, 3)$. MobileNetV2 converts the ten frames into ten feature vectors, producing a recurrent input of $(10, D)$. The batch tensor therefore has shape $(B, 10, D)$.

In the execution, $D = 1280$, so the extracted feature tensor for one video was $(10, 1280)$ and the full video feature dataset was split into tensors with the form $(B, 10, 1280)$.

The first CNN-LSTM test sample was classified as Archery with confidence 0.9980. The CNN-GRU test predictions also showed high confidence; the observed test classification report was perfect on all 19 test videos.

20. CNN Feature Extraction

MobileNetV2 was loaded with ImageNet weights, the classification head was removed using `include_top=False`, global average pooling was enabled, and the CNN was frozen before feature extraction.



The observed CNN output dimension was 1280, and the frame tensor (10, 224, 224, 3) was transformed into (10, 1280).

21. CNN-LSTM / CNN-GRU Model

Both models used 10-frame sequences, 32 recurrent units, dropout 0.2, Dense(16, ReLU) and a three-class softmax output. The pretrained MobileNetV2 feature extractor remained frozen.

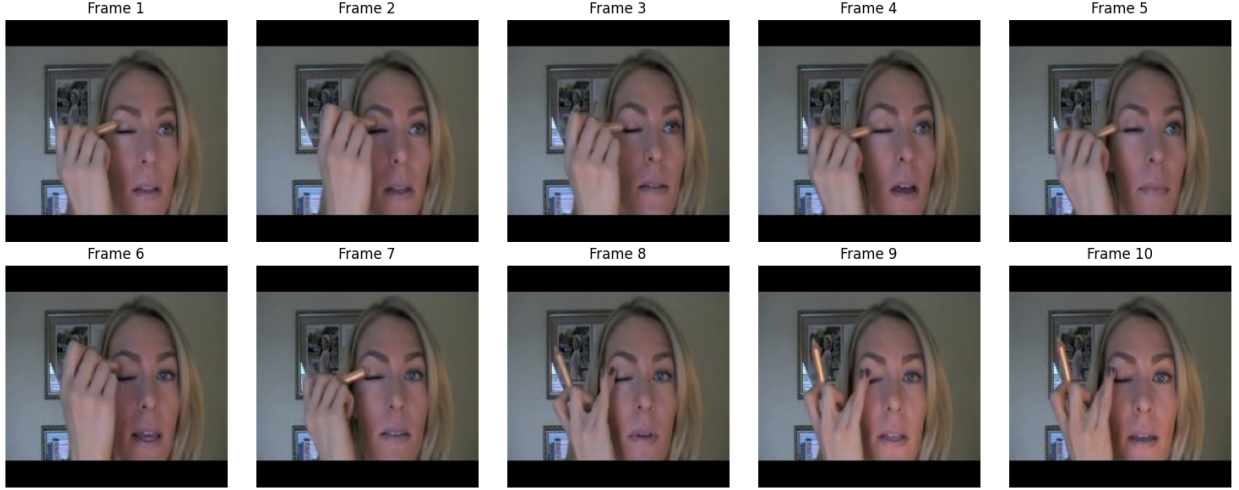


Figure 15: Ten sampled frames from a representative video.

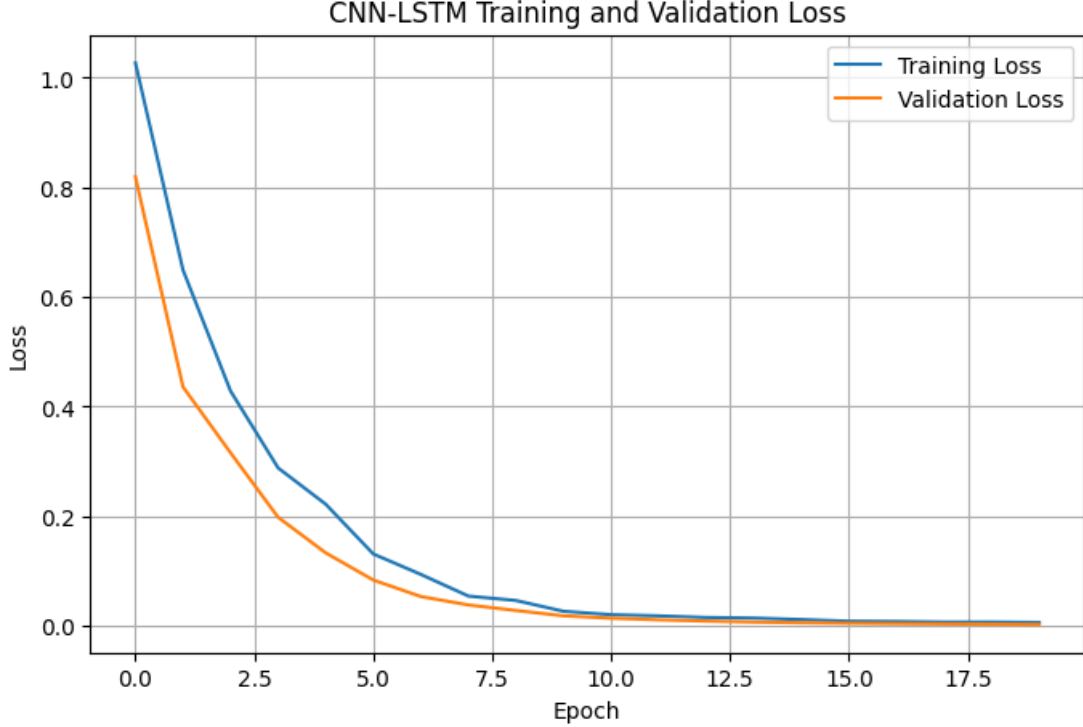


Figure 16: CNN-LSTM training and validation loss.

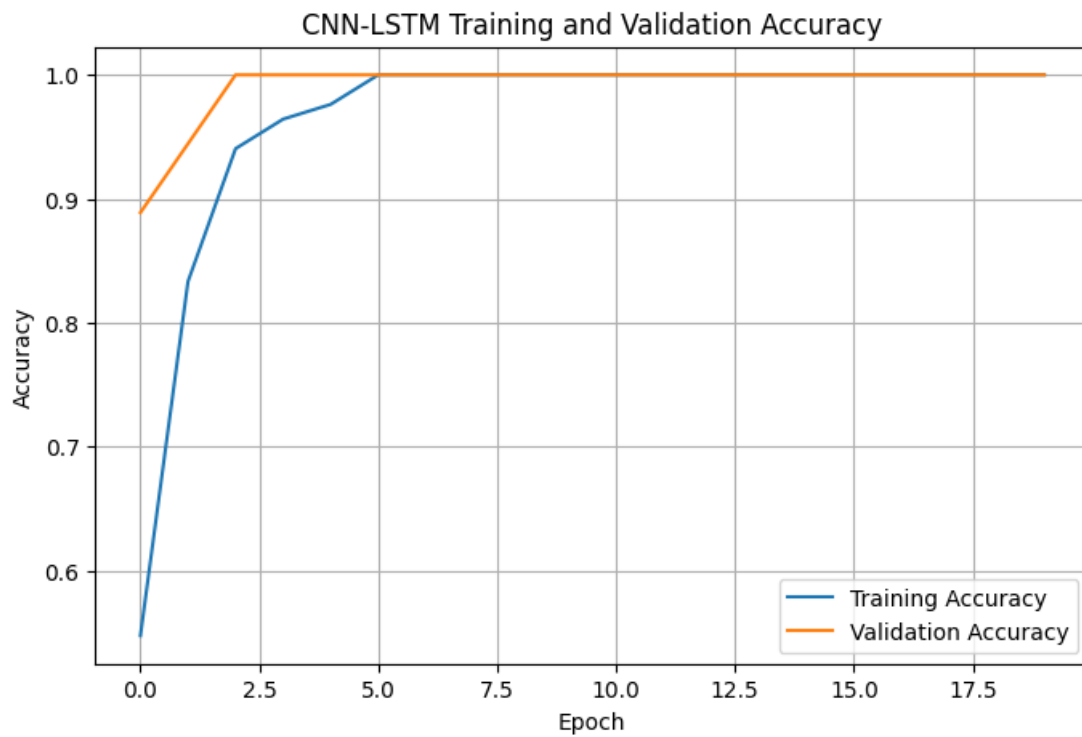


Figure 17: CNN-LSTM training and validation accuracy.

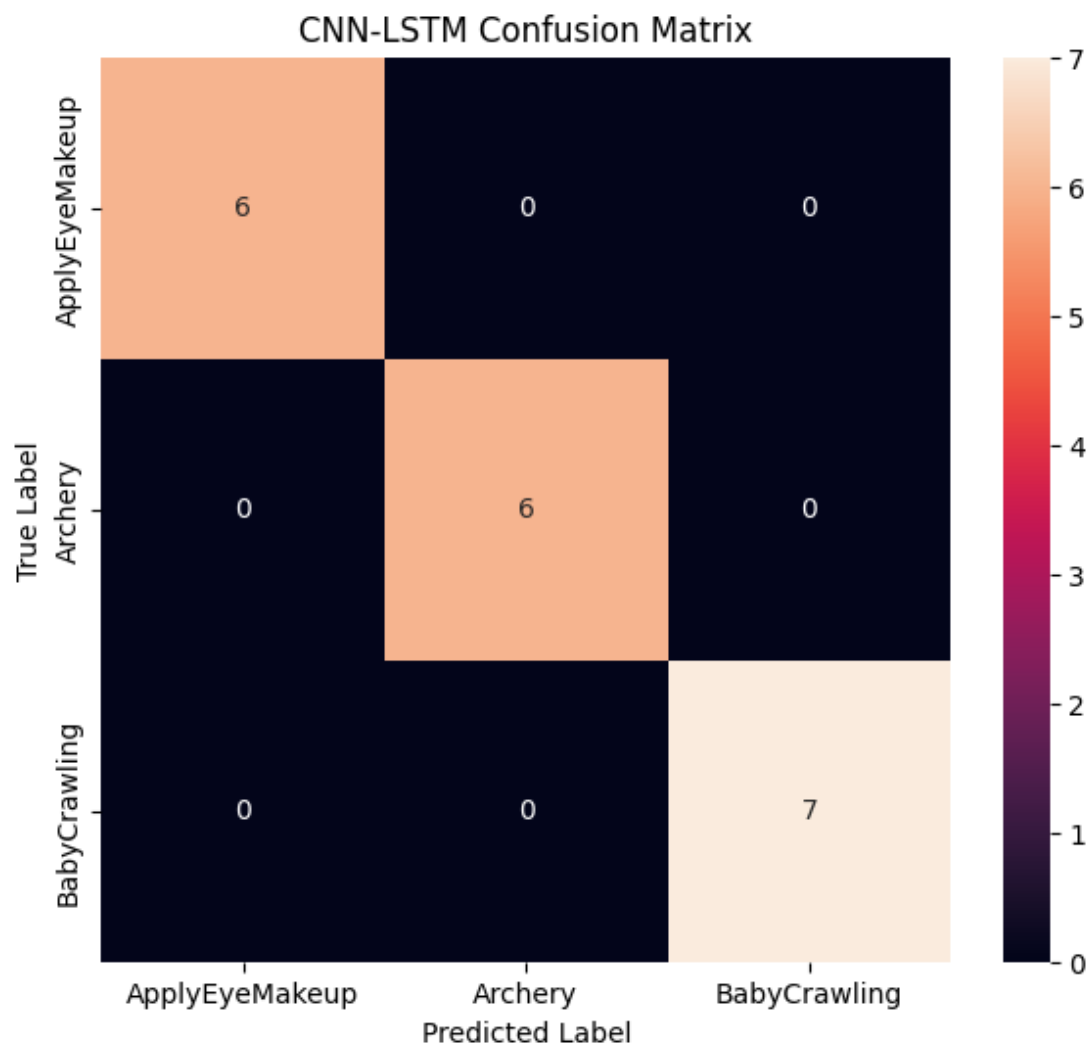


Figure 18: CNN-LSTM confusion matrix.

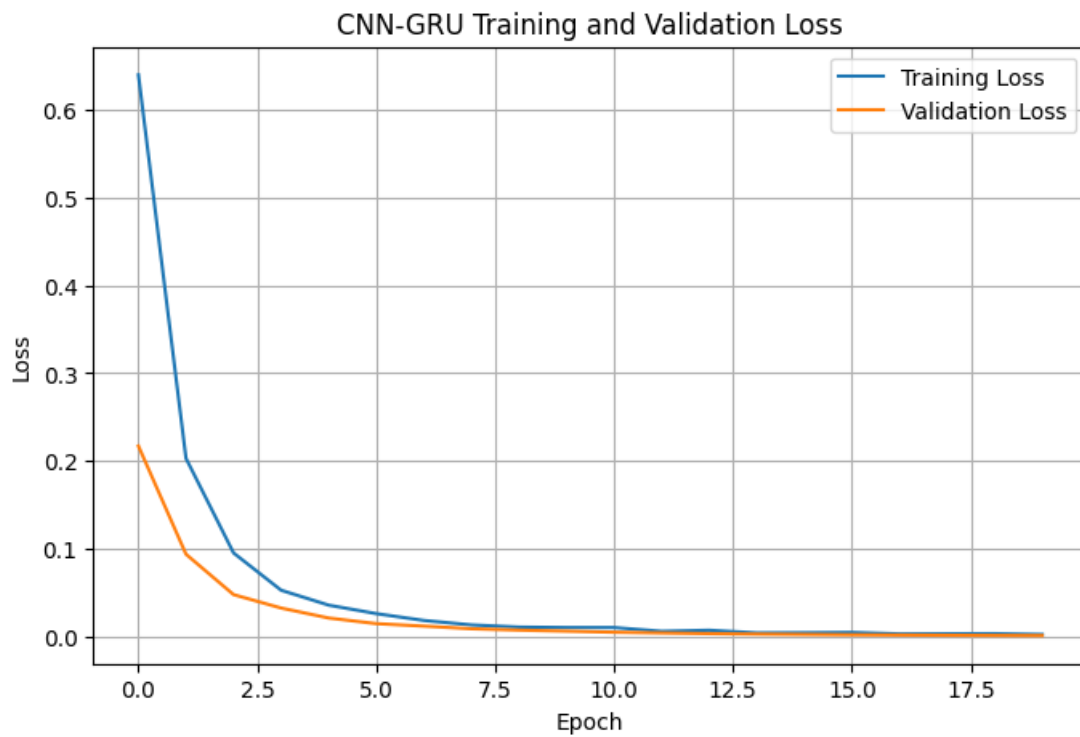


Figure 19: CNN-GRU training and validation loss.

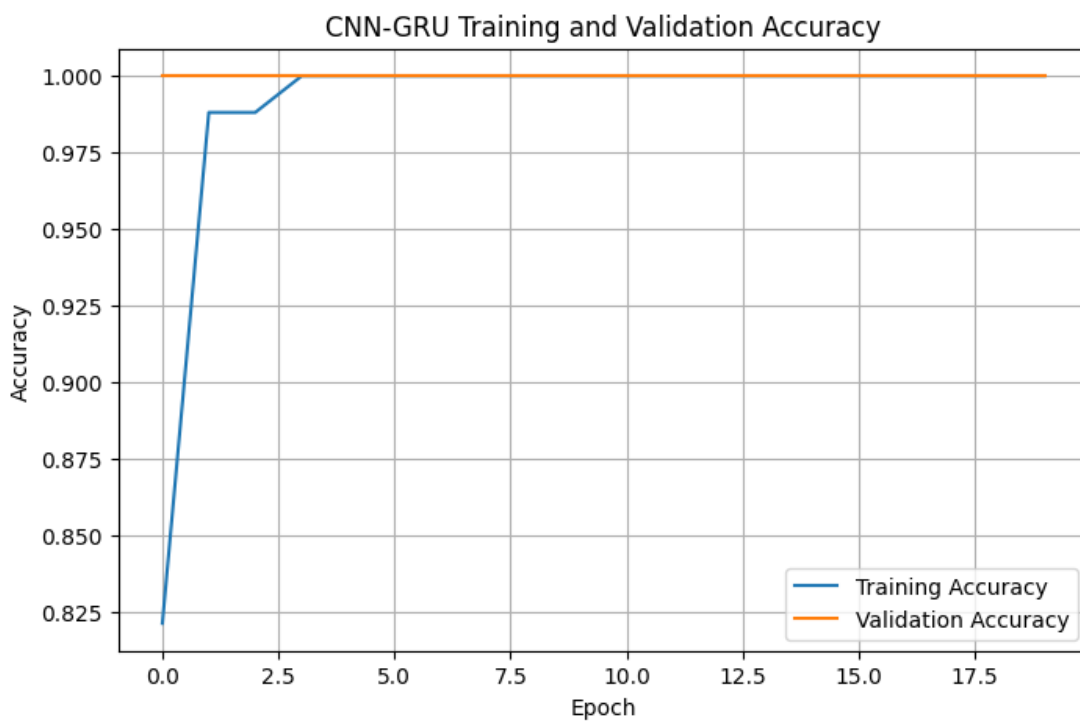


Figure 20: CNN-GRU training and validation accuracy.

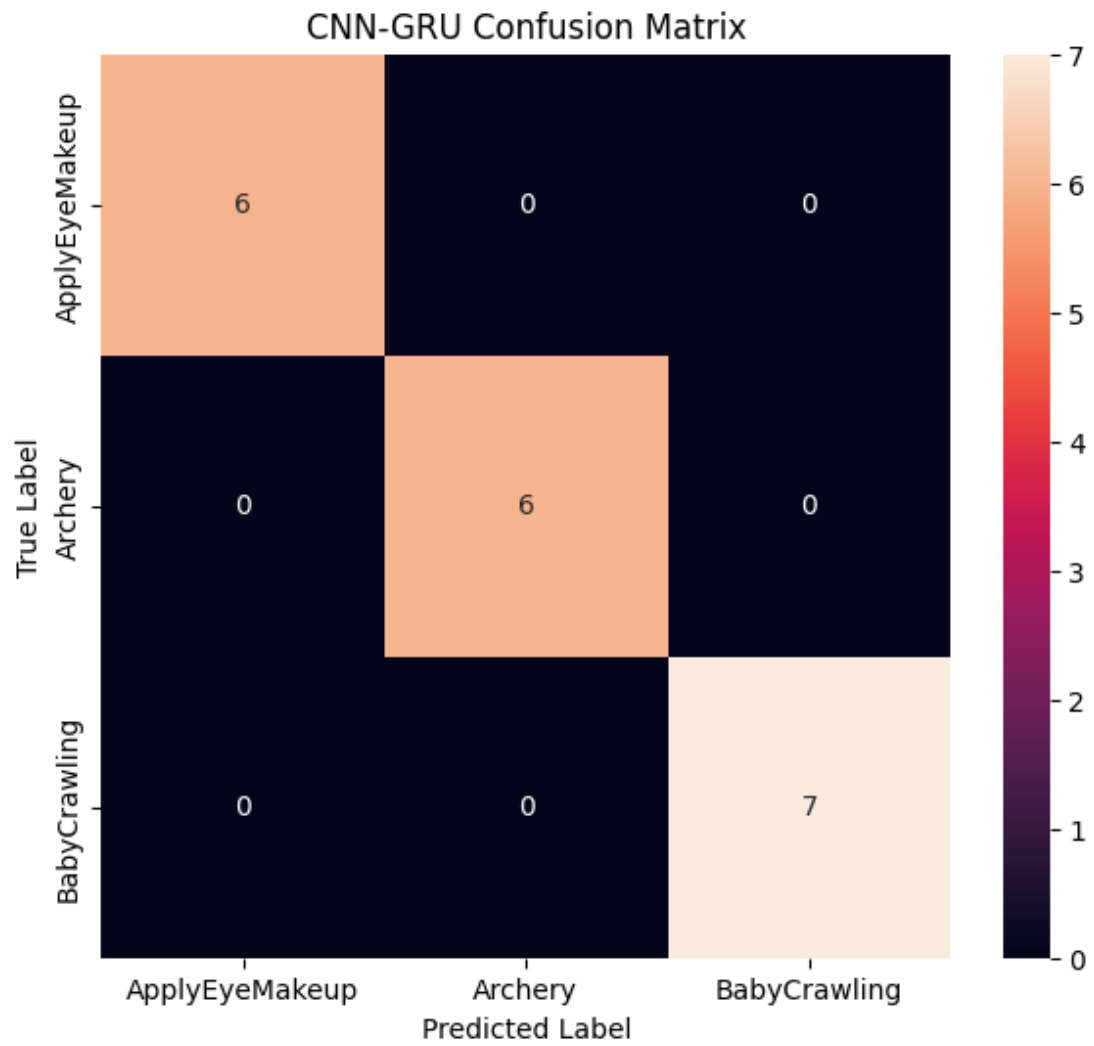


Figure 21: CNN-GRU confusion matrix.

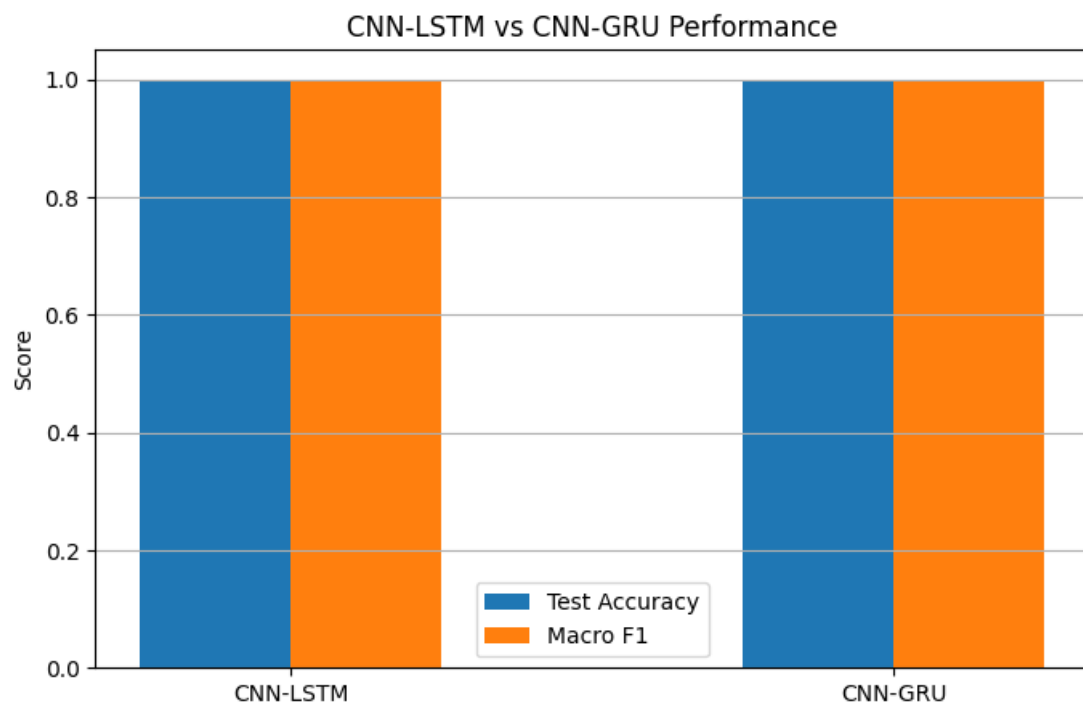


Figure 22: CNN-LSTM versus CNN-GRU performance.

Model	Accuracy	Precision	Recall	F1	Parameters	Time (s)
CNN-LSTM	100.00%	100.00%	100.00%	100.00%	168,643	5.85
CNN-GRU	100.00%	100.00%	100.00%	100.00%	126,723	5.50

Inference. The CNN extracts spatial information from individual frames, such as visual patterns useful for distinguishing the selected actions. The recurrent layer receives the sequence of CNN features and models how those spatial representations evolve over time. Both models achieved perfect classification on the 19-video test split in this execution. Because the video subset is small, the observed 100% result should be interpreted as a result for this experiment rather than as evidence of state-of-the-art generalization.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps an input sequence to an output sequence. The laboratory task uses a synthetic reversal problem, for example

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

The encoder LSTM converts the input sequence into a learned context representation and the decoder LSTM generates the reversed sequence one token at a time.



The notebook generated 2000 sequences of length four, used 1600 for training and 400 for testing, and trained the encoder–decoder model for 20 epochs.

23. Expected Sequence-to-Sequence Input and Output

An observed example was

$$[7, 4, 8, 5] \rightarrow [5, 8, 4, 7].$$

The decoder input was shifted using a start token representation, so that the decoder learned to generate the target sequence one position at a time.

Example	Input	Expected Output	Predicted Output
1	[9,8,3,3]	[3,3,8,9]	[3,3,8,9]
2	[1,4,4,9]	[9,4,4,1]	[9,4,4,1]
3	[4,4,2,1]	[1,2,4,4]	[1,2,4,4]
4	[9,4,6,7]	[7,6,4,9]	[7,6,4,9]
5	[1,2,9,7]	[7,9,2,1]	[7,9,2,1]

24. Sequence-to-Sequence Evaluation

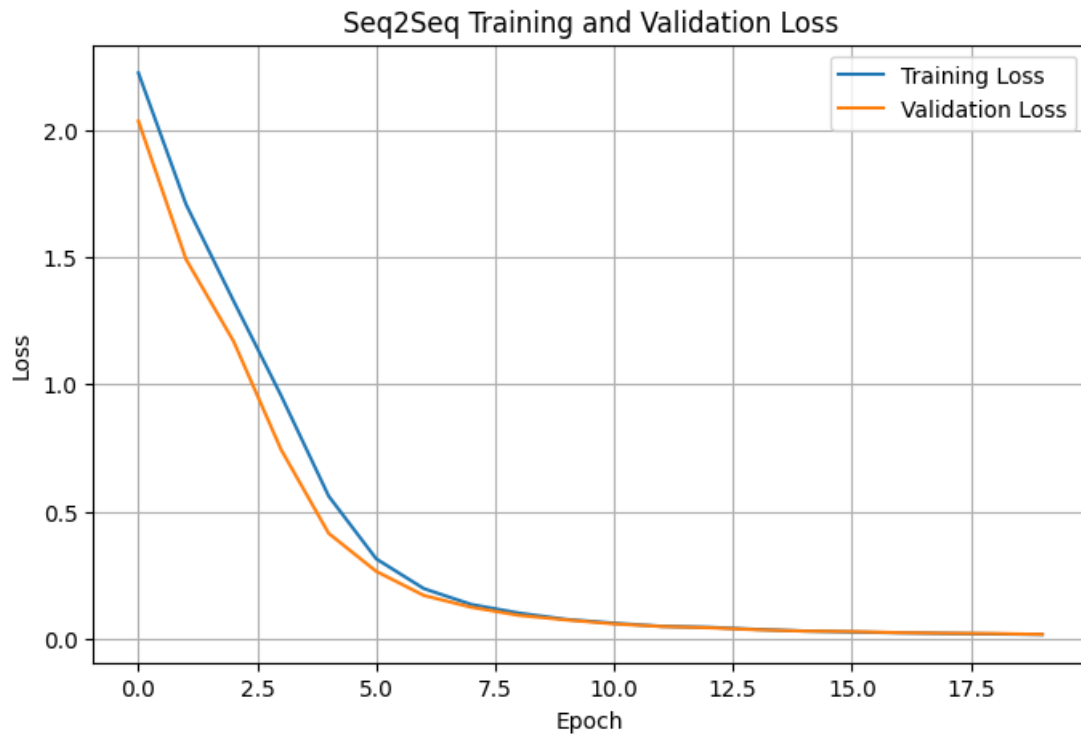


Figure 23: Sequence-to-sequence training and validation loss.

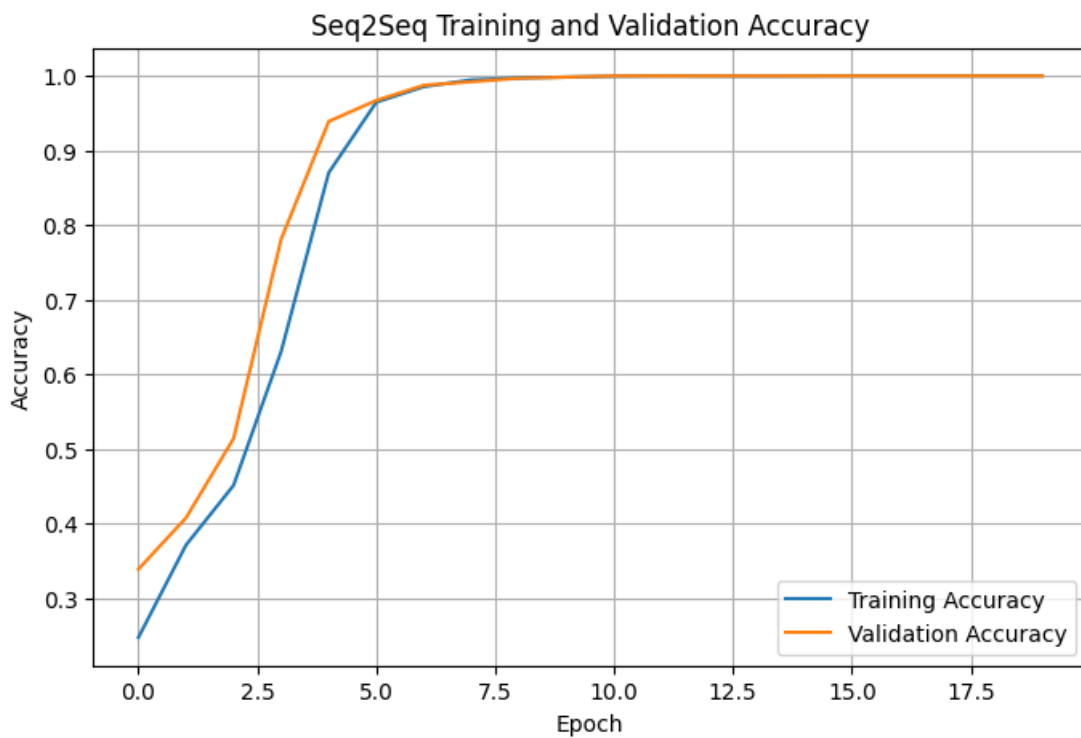


Figure 24: Sequence-to-sequence training and validation accuracy.

The observed results were:

Metric	Observed value
Token accuracy	99.9375%
Sequence accuracy	99.7500%
Training time	13.46 s
Trainable parameters	50,954

Token accuracy measures individual token predictions. Sequence accuracy is stricter because an entire sequence must be correct. The observed difference was 0.0019 (0.19 percentage points), showing that a small number of token-level errors can cause complete-sequence accuracy to be lower.

25. Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters	Time (s)
Vanilla RNN	84.08%	83.65%	83.50%	83.49%	1,974	80.37
LSTM	95.02%	95.37%	95.38%	95.34%	6,006	79.51
GRU	95.08%	95.46%	95.41%	95.38%	4,758	73.99
CNN-LSTM	100.00%	100.00%	100.00%	100.00%	168,643	5.85
CNN-GRU	100.00%	100.00%	100.00%	100.00%	126,723	5.50

Sequence-to-sequence metric	Value
Token accuracy	99.9375%
Sequence accuracy	99.7500%
Training time	13.46 s
Trainable parameters	50,954

26. Required Inferences

1. **Temporal pattern:** Dynamic activities show stronger temporal variation than SITTING, where the selected signals are relatively stable.
2. **RNN convergence:** The Vanilla RNN improves gradually but has noticeable fluctuations in its training curves and lower final test performance.
3. **LSTM convergence:** LSTM reaches high validation accuracy rapidly and maintains a stable low-loss regime.
4. **GRU convergence:** GRU also converges quickly and maintains high accuracy with fewer parameters than LSTM.
5. **Overfitting/underfitting:** The RNN curves show a larger generalization fluctuation. The gated models have smaller train/validation gaps for most of training, although individual validation fluctuations occur.
6. **Activity-wise errors:** The most persistent confusion across recurrent models is SITTING versus STANDING, while LAYING is recognized extremely well.
7. **Sequence length:** RNN benefits progressively from longer sequences in this run; GRU also improves toward 128; LSTM peaks at 64 and drops at 128.
8. **Parameters and computation:** RNN has 1,974 parameters, LSTM 6,006 and GRU 4,758. GRU also has the lowest training time among the three recurrent models in this run.
9. **CNN role:** MobileNetV2 converts each frame into a 1280-dimensional spatial feature vector.
10. **Recurrent role in video:** LSTM/GRU process the ordered sequence of frame features to learn temporal information.
11. **Token versus sequence accuracy:** Sequence accuracy is lower because every token in a sequence must be correct simultaneously.

27. Discussion Questions

1. **What is an RNN?** A neural network that maintains a recurrent hidden state while processing an ordered sequence.
2. **What does the hidden state represent?** A learned summary of information from the current input and preceding sequence context.
3. **Why is sequence order important?** The meaning of a sequential signal depends on how observations evolve over time.

4. **Feed-forward network vs RNN:** A feed-forward network processes inputs without recurrent state, whereas an RNN explicitly carries information across time steps.
5. **BPTT:** BPTT unrolls the recurrent network over time and propagates the loss gradient backward through the unrolled states.
6. **Why do gradients vanish?** Repeated multiplication by derivatives with magnitude below one can shrink gradients exponentially across many time steps.
7. **Why do gradients explode?** Repeated multiplication by factors larger than one can make gradients grow to very large magnitudes.
8. **Long-term dependencies:** Relationships where information from much earlier time steps is useful for interpreting later observations.
9. **LSTM cell state:** It provides a persistent memory pathway whose information is controlled by learned gates.
10. **LSTM gates:** Forget controls retained cell information, input controls newly written information, and output controls exposed hidden information.
11. **GRU gates:** The reset gate controls how much previous state contributes to a candidate state, while the update gate controls the interpolation between old and candidate state.
12. **LSTM vs GRU:** LSTM has a separate cell state and three main gates; GRU uses a single hidden state and two main gates.
13. **Why is GRU simpler?** It combines memory and hidden representation and uses fewer gating components.
14. **Why same experimental settings?** To make differences in measured performance more attributable to the recurrent architecture rather than different training conditions.
15. **What does a confusion matrix reveal?** It shows class-specific correct predictions and the particular classes being confused.
16. **Why macro F1?** It gives equal weight to each class by averaging class-wise F1 values, which is useful for multi-class evaluation.
17. **Significance of sequence length:** It determines how much temporal context is available to the recurrent model.
18. **Why use CNN as a video feature extractor?** CNNs efficiently transform image frames into learned spatial representations.
19. **What does the CNN extract?** Spatial visual features such as shapes, textures and higher-level image patterns useful for classification.
20. **What does the recurrent model learn?** Temporal relationships among the frame-level feature vectors.
21. **Why arrange CNN features as a sequence?** The order of frames carries temporal information that a recurrent model can exploit.
22. **Encoder and decoder:** The encoder maps the input sequence to a learned representation and the decoder generates the output sequence from that representation.
23. **Teacher forcing:** During training, the decoder can receive the ground-truth previous token rather than its own previous prediction.
24. **Token vs sequence accuracy:** Token accuracy evaluates individual positions; sequence accuracy requires every position of an entire sequence to be correct.
25. **Why keep the test set untouched?** To preserve an unbiased estimate of generalization after model and hyperparameter choices have been made.

28. Additional Exercises

The laboratory specification proposes the following extensions for further study:

1. replace 32 recurrent units with 16 and 64 and compare accuracy, F1, parameter count and time;
2. compare GRU and LSTM with the same number of recurrent units;
3. add a second recurrent layer;
4. compare bidirectional and unidirectional LSTM;
5. vary sequence length and measure computational cost;
6. compare LSTM and GRU after extracting identical CNN features; and
7. modify the seq2seq task so that output length differs from input length.

These exercises were listed in the supplied laboratory specification and were not included in the executed notebook results supplied for this report.

29. Expected Outcome

The experiment demonstrates the complete sequence-learning pipeline

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline

Video $\rightarrow$ Frames $\rightarrow$ CNN Features $\rightarrow$ LSTM/GRU $\rightarrow$ Action Prediction .

The sequence-to-sequence formulation is

Input Sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ Output Sequence .

The executed results demonstrate that gated recurrent architectures substantially improved the HAR classification metrics over the Vanilla RNN in this run. The video experiment successfully converted frame sequences into CNN feature sequences and classified the three selected UCF101 classes. The synthetic seq2seq experiment successfully learned sequence reversal with very high token and sequence accuracy.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>

31. Source Code

Github Link: <https://github.com/mokshithreddynarapureddy/DeepLearningLab/tree/main/Experiment-6>